

Instalación y configuración de soluciones de alta disponibilidad. Virtualización

Seguridad y Alta Disponibilidad

Ventajas de la virtualización

. Ahorro de costos

- Reducción en la compra de hardware físico, ya que un único servidor puede ejecutar varias máquinas virtuales (VMs).
- Disminución del consumo energético y los costes de refrigeración en centros de datos.
- Reducción de los gastos de mantenimiento, ya que menos equipos físicos significan menos averías y reemplazos.

Optimización del uso de los recursos

- Permite utilizar de forma más eficiente la capacidad de CPU, RAM y almacenamiento.
- Distribuye dinámicamente los recursos según la demanda de cada máquina virtual.
- Mejora la escalabilidad, ya que se pueden agregar más VMs sin necesidad de hardware adicional inmediato.

• Facilidad de administración y despliegue

- Gestión centralizada de todas las VMs a través de herramientas como VMware vSphere, Microsoft Hyper-V o Proxmox.
- Creación y eliminación rápida de entornos de prueba o desarrollo sin necesidad de adquirir nuevos equipos.
- Implementación rápida de servidores y servicios sin depender de infraestructura física.

• Seguridad y aislamiento

- Cada VM está aislada de las demás, lo que minimiza los riesgos de seguridad en caso de ataques o fallos del sistema.
- Posibilidad de realizar copias de seguridad y snapshots para recuperación rápida ante fallos.
- Mayor capacidad de implementar políticas de seguridad y control de acceso.

Ventajas de la virtualización

Compatibilidad y flexibilidad

- Posibilidad de ejecutar sistemas operativos diferentes en un mismo hardware (Windows, Linux, BSD, etc.).
- Compatibilidad con arquitecturas antiguas sin necesidad de mantener hardware obsoleto.
- Virtualización en la nube, permitiendo acceso desde cualquier ubicación

Alta disponibilidad y tolerancia a fallos

- Se pueden configurar sistemas de alta disponibilidad (HA) que reubican automáticamente las VMs en caso de fallo de un servidor físico.
- Uso de soluciones como balanceo de carga y migración en caliente (live migration) para evitar interrupciones en los servicios.

Mejor recuperación ante desastres (Disaster Recovery)

- Posibilidad de replicar VMs en diferentes ubicaciones para garantizar la continuidad del negocio en caso de fallos graves.
- Restauración rápida en caso de ataque de malware o pérdida de datos.

Soporte para entornos de desarrollo y pruebas

- Facilita la creación de entornos de pruebas aislados sin afectar la infraestructura productiva.
- Permite probar diferentes configuraciones sin riesgos.
- Reducción del tiempo de implementación de nuevos proyectos.

Tipos de Máquina Virtual según el Tipo de Hypervisor

Máquinas Virtuales de Tipo 1 (Bare Metal)

También llamadas máquinas virtuales de hipervisor tipo 1, o simplemente "bare metal".

- Se ejecutan directamente sobre el hardware físico del servidor sin necesidad de un sistema operativo anfitrión.
- Ofrecen mayor rendimiento y menor latencia en comparación con las de tipo 2.
- Se utilizan en centros de datos y servidores empresariales donde la eficiencia y seguridad son cruciales.

Ejemplos de software (Hypervisores Tipo 1):

- VMware ESXi
- Microsoft Hyper-V (en modo bare metal)
- Xen
- KVM (Kernel-based Virtual Machine)
- Proxmox VE

Tipos de Máquina Virtual según el Tipo de Hypervisor

- **Ventajas:**

- Mayor rendimiento al ejecutarse directamente sobre el hardware.
- Mejor uso de los recursos del sistema.
- Mayor estabilidad y seguridad, ya que no depende de un sistema operativo anfitrión.
- Ideal para entornos de alta disponibilidad y cloud computing.

- **Desventajas:**

- Instalación y configuración más complejas.
- Menos flexible para usuarios de escritorio o desarrolladores que necesitan pruebas rápidas.

- **Casos de uso:**

- Infraestructura de servidores en centros de datos.
- Virtualización en la nube (AWS, Google Cloud, Azure).
- Entornos de producción donde la estabilidad y el rendimiento son prioritarios.

Tipos de Máquina Virtual por Niveles

Máquinas Virtuales de Tipo 2 (Sobre un Sistema Operativo)

- Se ejecutan sobre un **sistema operativo anfitrión** (Windows, Linux o macOS).
- El hipervisor se instala como un programa más dentro del SO.
- Son **más flexibles** y fáciles de usar, pero tienen **mayor sobrecarga** porque dependen del rendimiento del SO anfitrión.

Ejemplos de software (Hipervisores Tipo 2):

- VMware Workstation
- Oracle VirtualBox
- Microsoft Hyper-V (cuando se usa en Windows)
- Parallels Desktop (para macOS)

Tipos de Máquina Virtual por Niveles

- **Ventajas:**

- Fácil instalación y configuración.
- Permite ejecutar VMs sin necesidad de modificar el hardware.
- Ideal para pruebas y desarrollo en entornos de escritorio.

- **Desventajas:**

- Menor rendimiento que los hipervisores tipo 1, debido a la sobrecarga del sistema operativo anfitrión.
- Mayor latencia y posible inestabilidad si el SO anfitrión tiene problemas.

- **Casos de uso:**

- Pruebas y desarrollo de software en entornos de escritorio.
- Uso de múltiples sistemas operativos en una misma máquina (Windows en Linux o viceversa).
- Virtualización ligera para usuarios individuales o pequeñas empresas.

Comparación entre Máquinas Virtuales Tipo 1 y Tipo 2

Característica	VM Tipo 1 (Bare Metal)	VM Tipo 2 (Sobre un SO)
Ejecutado en	Directamente sobre el hardware	Sobre un sistema operativo
Rendimiento	Alto	Medio a bajo
Latencia	Baja	Alta
Facilidad de instalación	Difícil	Fácil
Casos de uso	Servidores, cloud computing, CPDs	Pruebas, desarrollo, entornos de escritorio
Ejemplos	VMware ESXi, KVM, Hyper-V (bare metal)	VirtualBox, VMware Workstation, Parallels

Tipos de Virtualización

Virtualización por Hardware (Virtualización de Servidor)

Se trata del método más común en entornos empresariales.

- Permite dividir un servidor físico potente en múltiples servidores virtuales más pequeños.
- Cada máquina virtual (VM) tiene su propio sistema operativo y recursos asignados.

Ejemplos de software:

- VMware ESXi
- Microsoft Hyper-V
- Proxmox VE
- KVM (Kernel-based Virtual Machine)
- **Ventajas:**
 - Mayor eficiencia en la utilización del hardware.
 - Reducción de costos operativos y energéticos.
 - Facilita la migración y el escalado de servicios.

Tipos de Virtualización

Virtualización de Sistema Operativo (Virtualización de Software)

- Se crean entornos virtuales dentro de un mismo sistema operativo, en lugar de virtualizar todo el hardware.
- Todas las máquinas virtuales comparten el mismo núcleo del SO anfitrión.

Ejemplos de software:

- Docker (para contenedores)
- LXC (Linux Containers)
- OpenVZ

- **Ventajas:**

- Uso eficiente de recursos, ya que no se emula el hardware completo.
- Mayor velocidad y rendimiento en comparación con la virtualización completa.
- Ideal para entornos de desarrollo y pruebas.

Tipos de Virtualización

Virtualización de Escritorio (VDI - Virtual Desktop Infrastructure)

- Permite que los usuarios accedan a su escritorio de trabajo desde cualquier lugar mediante una conexión remota.
- Todos los escritorios virtualizados se ejecutan en un servidor central.

Ejemplos de software:

- Citrix Virtual Apps & Desktops
- VMware Horizon
- Microsoft Remote Desktop Services (RDS)

- **Ventajas:**

- Seguridad mejorada al centralizar los datos en servidores corporativos.
- Reducción de costos en licencias y mantenimiento de equipos físicos.
- Facilita el teletrabajo y el acceso remoto.

Tipos de Virtualización

Virtualización de Red

- Se combinan los recursos físicos y lógicos de la red para presentarlos como una única unidad virtual.
- Facilita la administración, reduce el cableado y mejora la eficiencia en centros de datos.

Ejemplos de software:

- VMware NSX
 - Cisco ACI
 - Open vSwitch
-
- **Ventajas:**
 - Mayor flexibilidad en la gestión de redes.
 - Segmentación de redes sin necesidad de hardware adicional.
 - Optimización del tráfico y mejora en la seguridad.

Tipos de Virtualización

Virtualización de Almacenamiento

- Consolida múltiples dispositivos de almacenamiento físico en un único recurso virtual.
- Mejora la escalabilidad y la gestión del almacenamiento.

Ejemplos de software:

- VMware vSAN
- Ceph
- FreeNAS

- **Ventajas:**

- Permite gestionar el almacenamiento de forma más eficiente.
- Facilita la replicación y recuperación de datos.
- Compatible con tecnologías RAID y SAN.

Tipos de Virtualización

Virtualización de Memoria

- Permite gestionar la memoria RAM como si fuera un único recurso distribuido entre varias máquinas virtuales.
- Se usa en entornos donde la memoria es un recurso crítico.

Ejemplos de software:

- Kernel Samepage Merging (KSM) en Linux
- Ballooning de memoria en VMware y Hyper-V

Ventajas:

- Optimización del uso de la memoria RAM.
- Mayor flexibilidad en la asignación de recursos.
- Reducción de costos al evitar compras innecesarias de hardware.

Tipos de Virtualización

Virtualización Gráfica (GPU Virtualization)

- Permite compartir recursos de GPU entre múltiples máquinas virtuales o acceder a gráficos de alto rendimiento de forma remota.
- Se usa en diseño gráfico, modelado 3D y aplicaciones de inteligencia artificial.

Ejemplos de software:

- NVIDIA GRID
- AMD MxGPU
- VMware vGPU

Ventajas:

- Acceso a potencia gráfica sin necesidad de hardware especializado en cada equipo.
- Ideal para entornos de ingeniería, diseño y videojuegos en la nube.
- Mejora la eficiencia en el uso de GPUs costosas.

Tipos de Virtualización

Virtualización de Aplicaciones

- Permite ejecutar aplicaciones sin necesidad de instalarlas en cada equipo.
- Se alojan en un servidor central y se accede a ellas desde múltiples dispositivos.

Ejemplos de software:

- Citrix Virtual Apps
- Microsoft App-V
- VMware ThinApp

- **Ventajas:**

- Reducción del mantenimiento y las instalaciones locales.
- Mayor seguridad al evitar la ejecución local de software.
- Compatibilidad con diferentes sistemas operativos.

Alta Disponibilidad y Virtualización

- **Ventajas de la Virtualización en Alta Disponibilidad**
- **Alta disponibilidad de aplicaciones y servicios**
 - Garantiza que los servicios y aplicaciones sigan funcionando incluso ante fallos en el hardware o el software.
 - Facilita la replicación de entornos para minimizar el impacto de interrupciones.
- **Reparto de carga eficiente (Load Balancing)**
 - Distribuye automáticamente la carga de trabajo entre varios servidores físicos.
 - Evita sobrecargas que pueden provocar caídas del sistema.
- **Independencia y aislamiento de servicios y aplicaciones**
 - Cada máquina virtual (VM) opera de forma independiente, reduciendo el riesgo de fallos en cascada.
 - Se pueden ejecutar múltiples sistemas operativos y aplicaciones en un mismo servidor sin interferencias.
- **Recuperación en caso de desastre (Disaster Recovery)**
 - Las VMs pueden replicarse en servidores remotos para garantizar la continuidad del negocio.
 - Facilita la restauración rápida en caso de pérdida de datos o ataques cibernéticos.
- **Protección y gestión de escritorios corporativos (Virtual Desktop Infrastructure - VDI)**
 - Si un equipo físico falla, el usuario puede iniciar sesión en otro dispositivo sin perder su entorno de trabajo.
 - Mayor seguridad, ya que los datos se almacenan en el servidor central y no en dispositivos locales.
- **Flexibilidad y escalabilidad**
 - Permite asignar más recursos (RAM, CPU, almacenamiento) a las VMs de forma dinámica según la demanda.
 - Evita saturaciones y caídas en los sistemas al adaptarse en tiempo real.

Pruebas de Carga y Cargas Sintéticas

Las pruebas de carga son fundamentales para evaluar el rendimiento y la estabilidad de una aplicación o servicio bajo diferentes niveles de tráfico. Su objetivo es medir tiempos de respuesta, detectar cuellos de botella y garantizar una experiencia óptima para los usuarios.

Tipos de Pruebas de Carga

Pruebas de rendimiento: Backend vs Frontend

- **Backend:** Evalúa cómo responde el servidor a las solicitudes del usuario.
 - Analiza la capacidad de procesamiento del servidor.
 - Mide el tiempo que tarda en recuperar y enviar datos a los clientes.
- **Frontend:** Mide el tiempo de carga y visualización de los elementos en el navegador.
 - Evalúa la velocidad con la que se renderizan imágenes, scripts y estilos.
 - Se centra en la experiencia del usuario.

Ejemplo:

- Un usuario accede a una página web y espera que cargue en menos de **3 segundos**.
- Se analizan los tiempos de respuesta del servidor, los scripts de la web y la latencia de la red.

Tipos de Pruebas de Carga

Pruebas de carga basadas en protocolos

- Se enfocan en medir el rendimiento del **backend**, simulando tráfico real con solicitudes **HTTP/HTTPS**.
- **Casos de uso:**
 - Pruebas en servidores web para ver cómo gestionan múltiples peticiones simultáneas.
 - Análisis del comportamiento de APIs REST.

Herramientas recomendadas:

- **Apache JMeter** (soporta HTTP, HTTPS, FTP, SOAP, REST)
- **k6** (pruebas de rendimiento para APIs y sitios web)
- **Gatling** (enfocado en pruebas de alta concurrencia)

Tipos de Pruebas de Carga

Pruebas de carga en el navegador

- Evalúan el rendimiento del **frontend**, midiendo tiempos de carga y renderizado de elementos.
- **Métricas clave:**
 - Tiempo de carga de imágenes y estilos CSS.
 - Tiempos de ejecución de JavaScript.
 - Latencia entre la solicitud y la respuesta del servidor.

Herramientas recomendadas:

- **Google Lighthouse** (pruebas de velocidad web y accesibilidad)
- **WebPageTest** (análisis de rendimiento web)

Tipos de Pruebas de Carga

Pruebas de componentes

- Se enfocan en analizar el rendimiento individual de un componente o servicio dentro del sistema.

Ejemplo:

- Evaluar la base de datos midiendo cuántas consultas por segundo puede manejar antes de saturarse.
- Medir la capacidad de procesamiento de un servidor antes de llegar a su límite.

Herramientas recomendadas:

- SysBench (pruebas de CPU, memoria y base de datos)
- PostgreSQL pgbench (evaluación de rendimiento de bases de datos)

Tipos de Pruebas de Carga

Pruebas de extremo a extremo (End-to-End, E2E)

- Simulan el comportamiento de un usuario real para evaluar el **rendimiento global del sistema**.

Ejemplo:

- Un usuario inicia sesión, busca un producto y finaliza la compra.
- Se analiza el tiempo total de respuesta del sistema.

Herramientas recomendadas:

- **Selenium** (automatización de pruebas en navegadores)
- **Cypress** (pruebas E2E modernas para aplicaciones web)

Monitorización Sintética y Cargas Sintéticas

Qué es la monitorización sintética?

Consiste en generar tráfico de manera controlada y programada para evaluar el rendimiento sin afectar el tráfico real de usuarios.

Qué son las cargas sintéticas?

Son peticiones generadas artificialmente para evaluar el rendimiento sin depender de usuarios reales.

- **Ventajas:**

- Menos impacto en el sistema en comparación con pruebas de carga masivas.
- Permite simular tráfico en horarios no críticos.
- Útil para evaluar nuevos cambios antes de lanzarlos al público.

- **Herramientas recomendadas:**

- k6 (pruebas de carga de código abierto, ideal para APIs y microservicios)
- BlazeMeter (herramienta basada en JMeter para pruebas sintéticas)

Que es Docker?



- Docker es una plataforma de **virtualización basada en contenedores** que permite empaquetar, distribuir y ejecutar aplicaciones de manera eficiente y portátil.
- En lugar de usar máquinas virtuales (VMs) tradicionales que requieren un sistema operativo completo por cada instancia, Docker **comparte el kernel del sistema operativo** y ejecuta aplicaciones en contenedores aislados.
- Esto permite que las aplicaciones se ejecuten de manera **rápida, ligera y consistente**, independientemente del entorno en el que se desplieguen (desarrollo, pruebas o producción).

Para que sirve Docker

Facilitar el Desarrollo de Software

- Permite que los desarrolladores trabajen en un entorno idéntico al de producción.
- Evita el clásico problema de "en mi máquina funciona, pero en producción no".
- Automatiza la instalación de dependencias y configuraciones.
- Ejemplo: Un desarrollador puede usar un contenedor con Python y Flask ya configurados, en lugar de instalarlos manualmente en su sistema.
- Desplegar Aplicaciones en Producción
 - Facilita la ejecución de aplicaciones en servidores sin preocuparse por compatibilidad de dependencias.
 - Permite ejecutar múltiples versiones de una misma aplicación sin conflictos.
 - Mejora la escalabilidad gracias a su integración con Kubernetes y otras herramientas de orquestación.
- Ejemplo: Un sitio web basado en Node.js puede ejecutarse en un contenedor Docker sin importar si el servidor usa Linux o Windows.

Para que sirve Docker

- Crear Entornos de Pruebas y CI/CD
 - Ideal para entornos de integración continua y despliegue continuo (CI/CD).
 - Permite probar aplicaciones en entornos controlados antes de publicarlas.
 - Ahorra recursos al usar contenedores en lugar de máquinas virtuales completas.
- Ejemplo: Un equipo de desarrollo puede usar Jenkins con Docker para ejecutar pruebas automatizadas en cada cambio del código.

Mejorar la Seguridad y Aislamiento de Aplicaciones

- Cada contenedor es independiente y no interfiere con otros en el sistema.
- Evita que fallos en una aplicación afecten al resto del sistema.
- Facilita la implementación de microservicios, donde cada servicio se ejecuta en su propio contenedor.
- Ejemplo: Una aplicación que usa MySQL, Redis y una API en Python puede ejecutarse en tres contenedores separados sin conflictos.

Facilitar la Portabilidad entre Sistemas

- Un contenedor creado en un sistema Linux puede ejecutarse sin modificaciones en Windows o macOS.
- Evita problemas de compatibilidad entre distintas versiones de software.
- Ejemplo: Un desarrollador puede crear una aplicación en su ordenador con Ubuntu y desplegarla en un servidor con CentOS sin problemas.